Finance

Accessing real-world data using Chainlink Data Feeds on Xhavic Blockchain

This tutorial demonstrates how developers can access secure, decentralized real world data (such as cryptocurrency prices) on Xhavic Blockchain, an OP Stack–based Layer 2 blockchain, using Chainlink Data Feeds.

Objectives

- Set up a smart contract project for Xhavic Blockchain
- Install Chainlink smart contracts
- Consume Chainlink price feeds
- Deploy contracts on Ethereum Sepolia (L1) and Xhavic L2
- Read prices on Xhavic Blockchain

Prerequisites

- Node.js v18+
- Hardhat
- MetaMask / Coinbase Wallet
- Sepolia ETH and Xhavic test ETH

Oracle Architecture

- Chainlink Data Feeds
- L1 Oracle Contract (Sepolia)
- Relayer Script
- L2 Oracle Contract (Xhavic Blockchain)
- DeFi / Trading Applications

Step 1: Create Project

```
mkdir xhavic-oracle cd
xhavic-oracle
npm init -y
npm install --save-dev hardhat
 npx hardhat init
```

Step 2: Install Chainlink Contracts

```
npm install @chainlink/contracts
```

Step 3: L1 Oracle Contract (Sepolia)

```
pragma solidity ^0.8.28;

interface AggregatorV3Interface {
function latestRoundData()
```

```
external view
returns (uint80, int256, uint256, uint256, uint80);
}

contract MultiPriceOracleL1 {
mapping(bytes32 => address) public feeds;
mapping(bytes32 => int256) public prices;

function addFeed(bytes32 symbol, address feed)
external { feeds[symbol] = feed;
}

function fetchPrice(bytes32 symbol) external {
(, int256 price,,,) =
AggregatorV3Interface(feeds[symbol]).latestRoundData();
prices[symbol] = price;
}

function getPrice(bytes32 symbol) external view returns (int256) {
return prices[symbol];
}
}
```

Step 4: L2 Oracle Contract (Xhavic)

```
pragma solidity ^0.8.20;
contract MultiPriceOracleL2 {
mapping(bytes32 => int256) public prices;
address public updater;

constructor(address _updater) {
updater = _updater;
}

function updatePrice(bytes32 symbol, int256 price) external {
require(msg.sender == updater, "Not authorized");
prices[symbol] = price;
}
}
```

Step 5: Deploy Contracts

L1:

```
npx hardhat run scripts/deploy-l1.js --network sepolia
```

L2:

```
npx hardhat run scripts/deploy-l2.js --network customL2
```

Step 6: Add Feed Step (Important)

Before fetching prices, developers must register the Chainlink feed address using addFeed. This step links a token symbol to an existing Chainlink price feed deployed on Ethereum L1. This is a one-time configuration per asset.

L1 – Register Feed

```
npx hardhat console --network sepolia

const oracle = await ethers.getContractAt(
  "MultiPriceOracleL1",
  process.env.L1_ORACLE
);

oracle.addFeed(ethers.encodeBytes32String("ETH"), ETH_FEED_ADDRESS);
oracle.addFeed(ethers.encodeBytes32String("BTC"), BTC_FEED_ADDRESS);
```

Step 7 : Relay Prices

```
node scripts/relay-price.js
```

Step 7: Read Prices on Xhavic

```
npx hardhat console --network customL2
const l2 = await ethers.getContractAt(
  "MultiPriceOracleL2",
  process.env.L2_ORACLE
);
(await l2.getPrice(ethers.encodeBytes32String("ETH"))).toString();
```

Conclusion

Xhavic Blockchain follows the same oracle architecture as Base, Optimism, and Arbitrum. Developers can safely build trading and DeFi applications using Chainlink secured price data.